

Table des matières

Cours 07 — Traitement distribué avec Spark	1
Objectifs	2
1. Ce qu’une machine fait déjà	2
2. Le modèle de Spark : un pilote, des exécuteurs, des partitions	3
3. Paresseux : transformations et actions	4
4. Le brassage, le seul geste vraiment cher	4
5. Lire le plan d’exécution	5
6. Ce qu’un moteur distribué écrit	6
7. Trois pièges qui coûtent tout	6
8. Quand une machine ne suffit plus	7
Points clés à retenir	8
Exercices	8

Cours 07 — Traitement distribué avec Spark

Théorie · Module 7/12 — Formation Data Engineering · ~40 min · intermédiaire

Où vous êtes : source → ingestion → **lac** → entrepôt → restitution. Le Cours 06 a confié le pipeline à un ordonnanceur. Ce cours-ci pose la question d’avant : ce que le travail demande comme machine. Il montre le moteur distribué que tout le monde nomme, Spark, sur les données de Boréal, et il mesure ce qu’il coûte face au moteur d’une seule machine employé depuis le Cours 02.

En 30 secondes :

- Un moteur distribué découpe les données en partitions, confie chaque partition à un exécuteur, et rassemble. La partition est son unité de travail, et presque tout s’explique par elle.
- Spark est paresseux. Une transformation ne calcule rien : elle allonge un plan. Une action déclenche tout ce que le plan contient.
- Le brassage est le seul geste vraiment cher. Il redistribue les lignes entre partitions, par le disque et le réseau, et il apparaît dès qu’on groupe, qu’on joint ou qu’on ordonne.
- Le plan d’exécution dit ce qui va se passer avant que ça se passe. Il se lit de bas en haut.

- Sur l'historien complet de Boréal, 2 407 796 lignes, le même travail prend 2,6 s sur une machine et 52 à 80 s en distribué. Le distribué coûte d'abord et rapporte ensuite, quand il y a de quoi.
- Le besoin d'une grappe ne se mesure pas en lignes, mais au moment où une machine ne tient plus le jeu de données ni le temps qu'on s'est donné.

L'idée du cours. Un cours sur Spark qui finirait par « installez une grappe » vous rendrait un mauvais service. Boréal produit 2,4 millions de points d'historien en deux ans, et son usine en produirait vingt fois plus qu'une machine suffirait encore. Ce cours vous fait donc écrire le même traitement des deux côtés, mesurer les deux, et lire les deux plans. Vous saurez employer Spark, parce que c'est ce que le marché demande, et vous saurez dire quand il n'a rien à faire là, ce qui vaut plus cher en entretien.

Objectifs

- Nommer les pièces d'un moteur distribué : pilote, exécuteurs, tâches, partitions.
- Expliquer ce qu'est une partition, d'où vient leur nombre, et pourquoi il commande tout le reste.
- Distinguer une transformation d'une action, et dire ce que l'évaluation paresseuse change.
- Reconnaître un brassage dans un plan d'exécution, et dire ce qu'il coûte.
- Lire un plan d'exécution de bas en haut, des deux côtés, et y retrouver les filtres poussés.
- Reconnaître les trois pièges classiques : le brassage inutile, la partition déséquilibrée, et l'action qui ramène tout au pilote.
- Décider, sur des critères mesurables, si un traitement a besoin d'un moteur distribué.

1. Ce qu'une machine fait déjà

Labo : Labo_07.md, étapes 1 et 2.

Avant de distribuer quoi que ce soit, il faut savoir ce qu'une seule machine fait. C'est la mesure que personne ne prend, et c'est pourtant la seule qui tranche.

L'usine de Val-des-Aulnes a livré son historien complet : 821 fichiers quotidiens, 2 407 796 lignes, 117 Mo de CSV. Sur le poste de l'auteur, douze cœurs et 32 Go, DuckDB convertit tout cela en Parquet partitionné par jour en 2,6 secondes, et calcule les agrégats quotidiens par étiquette, 47 618 lignes, en moins de deux dixièmes de seconde.

Le travail	Sur une machine
lire 821 CSV et écrire le bronze en Parquet, un dossier par jour	2,6 s

Le travail	Sur une machine
agréger 2 405 366 points en 47 618 lignes, une par étiquette et par jour	0,17 s
dédoublonner 204 485 messages de télémétrie par une fonction de fenêtrage	0,30 s

Ces chiffres ne sont pas une curiosité. Ils sont le point de comparaison de tout le cours, et la raison pour laquelle sa conclusion est écrite d'avance. Un moteur d'une machine moderne lit le Parquet en colonnes, pousse les filtres jusqu'au fichier, et occupe tous les cœurs sans qu'on le lui demande. Ce qui coûtait une grappe il y a dix ans tient aujourd'hui sur un portable.

À retenir — la première mesure est celle du moteur que vous avez déjà. Un projet qui commence par installer un moteur distribué ne sait pas ce qu'il achète, parce qu'il n'a jamais mesuré ce qu'il avait. La question n'est pas « Spark est-il rapide », elle est « qu'est-ce qui ne tient pas ici ».

2. Le modèle de Spark : un pilote, des exécuteurs, des partitions

Labo : Labo_07.md, étape 3.

Spark est un moteur qui suppose plusieurs machines, même quand il n'en a qu'une. Trois pièces, et une unité de travail.

La pièce	Ce qu'elle fait
le pilote	exécute votre programme, construit le plan, découpe le travail en tâches et les distribue ; c'est lui qui reçoit les résultats que vous demandez
les exécuteurs	des processus qui font le travail, chacun sur sa part des données, et qui gardent en mémoire ce qu'on leur dit de garder
le gestionnaire de grappe	attribue les machines aux exécuteurs ; en local, c'est Spark lui-même
la partition	la part de données qu'une tâche traite, du début à la fin, sans jamais regarder les autres

La partition est le mot à retenir. Un traitement Spark est une suite de tâches, une par partition, et tout ce qui va bien ou mal se ramène à leur nombre et à leur taille. Trop peu de partitions, et les cœurs attendent. Trop, et on paie l'ouverture de chacune. Inégales, et la plus grosse impose sa durée à tout le lot.

Leur nombre ne se devine pas, il se calcule à partir de deux réglages, relevés à l'essai du cours : une partition vise 128 Mo (`spark.sql.files.maxPartitionBytes`), et ouvrir un fichier compte pour 4 Mo (`spark.sql.files.openCostInBytes`). Les 821 fichiers de l'historien pèsent 145 Ko en moyenne, donc leur coût d'ouverture domine leur taille : Spark en met une trentaine par partition, et la lecture démarre avec 27 partitions pour 12 cœurs.

Analogie — la salle de tri. Le pilote est le chef de quai : il ne touche pas un colis, il découpe la cargaison en lots et dit qui prend quoi. Les exécuteurs sont les trieurs. Une partition est un lot posé devant un trieur. Donner trente lots à douze trieurs fonctionne ; en donner un seul, énorme, fait travailler une personne pendant que onze regardent.

En local, le pilote et les exécuteurs vivent dans le même processus, et `local[*]` prend tous les cœurs de la machine, douze à l’essai du cours. C’est un vrai Spark, avec le même plan et le même brassage qu’en grappe, sans le réseau. C’est exactement ce qu’il faut pour apprendre, et le labo n’installe donc aucune grappe.

3. Paresseux : transformations et actions

Labo : *Labo_07.md*, étape 3.

Le code qui décrit le travail ne le fait pas. Une transformation (`filter`, `select`, `groupBy`, `join`, `withColumn`) rend un nouveau jeu de données sans rien lire : elle allonge le plan. Seule une action (`count`, `collect`, `show`, une écriture) déclenche le calcul, et alors tout le plan part d’un coup.

Ce que vous écrivez	Ce qui se passe
<code>df = spark.read.csv(...)</code>	rien, sauf la lecture du schéma quand on ne le déclare pas
<code>df.filter(...).groupBy(...).agg(...)</code>	rien : le plan s’allonge de trois nœuds
<code>df.write.parquet(...)</code>	tout : la lecture, le filtre, l’agrégat et l’écriture, en une fois

Deux conséquences pratiques, et elles surprennent les deux fois.

La première : une erreur de chemin ou de type n’apparaît pas là où vous l’avez écrite, mais à l’action suivante, parfois trente lignes plus bas. La pile d’appels montre l’action, pas la transformation fautive.

La seconde : deux actions sur le même jeu refont le travail deux fois. `df.count()` puis `df.write.parquet(...)` lit les fichiers deux fois, à moins de demander explicitement à Spark de garder le résultat en mémoire, par `cache()` ou `persist()`. Un moteur d’une machine qui a écrit sa table ne la recalcule pas ; Spark, lui, ne garde rien qu’on ne lui ait demandé de garder.

Piège — la mesure qui ne mesure pas ce qu’on croit. Chronométrer une ligne de transformation donne toujours quelques millisecondes, et c’est faux. On chronomètre l’action, et on nomme dans le journal ce qu’elle a déclenché. Le labo mesure ainsi chacun de ses traitements.

4. Le brassage, le seul geste vraiment cher

Labo : *Labo_07.md*, étapes 5 et 6.

Tant qu’une tâche travaille sur sa partition sans rien demander aux autres, un moteur distribué est efficace : chaque cœur lit sa part et avance. Cela vaut pour un filtre, une projection, un calcul de colonne. Ces opérations sont dites étroites, et elles ne coûtent que la lecture.

Le brassage commence dès qu'une ligne doit changer de partition. Grouper par étiquette exige que toutes les lignes d'une même étiquette se retrouvent ensemble. Joindre deux jeux exige que les clés égales se rencontrent. Ordonner globalement exige de comparer des lignes qui n'étaient pas voisines. Spark écrit alors chaque partition sur disque, redistribue les morceaux par le réseau, et relit.

Trois choses à savoir, et le labo les vérifie.

Le nombre de partitions après un brassage ne dépend plus des fichiers, mais d'un réglage : `spark.sql.shuffle.partitions`, qui vaut 200 par défaut. Ce nombre a été choisi pour des grappes, et il est absurde pour 47 618 lignes de résultat.

Spark corrige en partie ce défaut tout seul. L'exécution adaptative, active par défaut depuis Spark 3 (`spark.sql.adaptive.enabled`), regarde la taille réelle des partitions après le brassage et replie les petites ensemble. À l'essai du cours, les 200 partitions des agrégats ont été repliées en trois fichiers.

Un agrégat bien écrit brasse peu, parce que Spark agrège d'abord dans chaque partition, puis ne brasse que les résultats partiels. C'est visible dans le plan, sous la forme de deux `HashAggregate` qui encadrent un `Exchange`. Le premier réduit les 2,4 millions de points à quelques milliers de lignes par partition ; c'est cela, et non le moteur, qui rend l'opération tenable.

À retenir — ce qui coûte, ce n'est pas le calcul, c'est le déplacement. Une moyenne sur deux millions de lignes est gratuite à côté du fait de les remettre en ordre. Quand un traitement Spark est lent, cherchez les `Exchange` dans son plan avant de toucher au code.

5. Lire le plan d'exécution

Labo : Labo_07.md, étapes 2 et 5.

Le plan est la seule façon honnête de savoir ce qu'un moteur va faire. Les deux moteurs du cours en donnent un, et ils se lisent pareil : de bas en haut, la source d'abord, le résultat en dernier.

Voici celui des agrégats quotidiens, côté Spark, tel que `explain()` le rend :

```
== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
+- HashAggregate(keys=[TagName, jour], functions=[count(1), avg(Value)])
   +- Exchange hashpartitioning(TagName, jour, 200), ENSURE_REQUIREMENTS
      +- HashAggregate(keys=[TagName, jour], functions=[partial_count(1), partial_avg(Value)])
         +- Project [TagName, Value, cast(Timestamp as date) AS jour]
            +- Filter (isnotnull(quality) AND (quality = 192))
               +- FileScan parquet [...] PushedFilters: [IsNotNull(quality), EqualTo(quality,192)]
```

Cinq lignes à retenir dans ce bloc. `FileScan` lit le Parquet, et sa mention `PushedFilters` dit que le filtre sur la qualité descend jusqu'au fichier : les lignes écartées ne remontent jamais. `Project` calcule la colonne du jour. Le premier `HashAggregate` agrège dans chaque partition. `Exchange hashpartitioning` est le brassage, avec son nombre de partitions de sortie. Le second `HashAggregate` finit le travail sur les résultats partiels.

Côté DuckDB, le même traitement donne un plan plus court : le balayage du Parquet avec ses filtres, une projection, un HASH_GROUP_BY, une projection finale. Il n’y a pas d’échange, parce qu’il n’y a rien à déplacer : un seul processus, une mémoire partagée. L’absence d’Exchange n’est pas une optimisation, c’est la différence de nature entre les deux moteurs.

Le plan de requête d’un moteur SQL, ses opérateurs et ses estimations de cardinalité appartiennent au Cours 10 de la formation SQL avancé, qui les enseigne pour de bon. Ici, il suffit de savoir repérer la source, les filtres poussés et les échanges.

6. Ce qu’un moteur distribué écrit

Labo : Labo_07.md, étape 4.

Une différence surprend tout le monde la première fois : Spark n’écrit pas un fichier, il écrit un dossier. Chaque tâche écrit sa part, et le nom porte le numéro de la partition, un identifiant unique et le codec de compression.

	Le moteur d’une machine	Spark
le bronze de l’historien	821 dossiers de jour, un fichier data_0.parquet chacun, 14 Mo en tout	821 dossiers de jour, un fichier part-00000-<identifiant>.c000.snappy.parquet chacun, 24 Mo
les agrégats	un fichier de 487 Ko	trois fichiers, 569 Ko
la journée du 2025-10-28	22 277 octets	31 756 octets

L’écart de taille ne vient pas du contenu, qui est le même au chiffre près, mais du codec par défaut. DuckDB écrit en zstd quand on le lui demande, comme au Cours 02 ; Spark écrit en snappy sauf instruction contraire (spark.sql.parquet.compression.codec). Snappy décompresse un peu plus vite, zstd compresse nettement mieux. Sur un lac qu’on garde des années, la différence se paie tous les mois.

Le nombre de fichiers n’est pas un détail de présentation. Un dossier de dix mille petits fichiers coûte cher à lire, sur un objet distant comme en local : chaque ouverture est une requête. C’est le défaut le plus commun des lacs construits sans surveillance, et le Cours 10, sur les formats de table ouverts, montrera comment on le répare.

7. Trois pièges qui coûtent tout

Labo : Labo_07.md, étape 7.

Le premier est le brassage inutile. Redistribuer les données « pour bien faire » avant un agrégat qui va de toute façon brasser double le travail. À l’essai du cours, le même agrégat précédé d’un repartition sur l’étiquette met 17,2 s au lieu de 8,8 s. Le code a l’air plus soigné, et il est deux fois plus lent.

Le deuxième est la partition déséquilibrée. Partitionner par une clé dont les valeurs ne se valent pas donne des parts sans rapport entre elles. Les 58 étiquettes de l’usine en sont l’exemple parfait : le banc d’essai enregistre 242 202 points, l’état de marche du compresseur d’appoint 1 833, soit 132 fois moins.

Redistribuer sur l'étiquette donne 37 partitions non vides sur 58 demandées, la plus grosse à 247 879 lignes et la plus petite à 1 833. La durée du lot est celle de la plus grosse, et onze cœurs attendent le douzième.

Le troisième est l'action qui ramène tout au pilote. `collect()` rapatrie toutes les lignes dans la mémoire d'un seul processus, celui qui n'était censé que coordonner. À l'essai, ramener les 2 407 796 lignes de l'historien a pris 19,5 s et a tenu, parce que le jeu est petit ; sur dix fois plus, le pilote tombe avec une erreur de mémoire, et le message ne nomme pas le coupable. La parade tient en une phrase : une action rend un résultat, pas un jeu de données. On écrit, on compte, on agrège, et on ne ramène que ce qu'un humain peut lire.

Piège — `show()` n'est pas `collect()`, et c'est heureux. `show(20)` demande vingt lignes, et Spark s'arrête dès qu'il les a. `collect()` demande tout. Entre les deux, `take(n)` et `limit(n)` font ce que leur nom dit. Le premier réflexe d'inspection est `show`, jamais `collect`.

8. Quand une machine ne suffit plus

Labo : Labo_07.md, étape 8.

Le cours conclut que Boréal n'a pas besoin de Spark. Reste à dire d'où le besoin viendrait, parce que c'est cette frontière qu'un ingénieur doit savoir situer.

Le signe	Ce qu'il veut dire
le jeu ne tient plus en mémoire, même par morceaux	une machine peut traiter plus grand que sa mémoire, mais le disque devient le goulot
le temps imparti est dépassé alors que tous les cœurs travaillent	il n'y a plus rien à gagner sur une machine
la donnée vit déjà répartie, sur un stockage objet, et personne ne veut la rapatrier	le calcul se déplace vers la donnée, pas l'inverse
le traitement doit reprendre après la panne d'une machine, sans tout refaire	un moteur distribué rejoue les tâches perdues ; un processus unique recommence

Pour Boréal, prenons la mesure du besoin plutôt que de l'imaginer. Une seule usine et deux ans donnent 2,4 millions de points. Vingt usines sur dix ans, à la même cadence, en donneraient environ 240 millions, soit une centaine de gigaoctets en Parquet : c'est encore le terrain d'un bon moteur d'une machine, avec un partitionnement soigné. Le basculement arrive plus tôt pour d'autres raisons que le volume : quand plusieurs équipes veulent lire en même temps, quand l'entreprise a déjà une plateforme distribuée et que s'en écarter coûte plus que de l'employer, ou quand le traitement doit tourner à côté de la donnée d'un lac d'entreprise.

À retenir — la bonne réponse dépend de ce qui est déjà là. Savoir écrire du PySpark reste utile même quand le moteur d'une machine suffit, parce que l'employeur qui vous embauche a peut-être déjà une plateforme Spark, et parce que le vocabulaire des partitions, des brassages et des plans sert partout. Ce que ce cours vous donne, c'est de pouvoir dire pourquoi, avec des chiffres.

Variante — Snowpark, le même geste sur Snowflake. Snowpark offre une API de DataFrame en Python dont le code ressemble beaucoup à du PySpark, mais qui se traduit en SQL exécuté par l'entrepôt Snowflake. Les notions de ce cours tiennent, la partition en moins : c'est l'entrepôt qui décide du parallélisme, et la taille de l'entrepôt qui commande la facture. La plateforme elle-même appartient à la formation Snowflake.

Recoupements — ce que ce cours partage avec ses voisines. - SQL avancé. Le plan d'exécution de Spark et celui d'un moteur SQL résolvent le même problème, avec le même vocabulaire de balayage, de filtre poussé et d'agrégation. Le Cours 10 de SQL avancé possède la lecture des plans et les estimations de cardinalité ; ici, on n'en prend que de quoi repérer une source, un filtre et un échange.

Points clés à retenir

- Un moteur distribué découpe les données en partitions et confie une tâche à chacune. Leur nombre vient des fichiers et de deux réglages, et il commande la durée de tout le traitement.
 - Spark est paresseux : les transformations allongent un plan, seule une action l'exécute. Deux actions sur le même jeu refont le travail deux fois.
 - Le brassage redistribue les lignes entre partitions, et c'est le seul geste vraiment cher. Il sort de tout groupement, de toute jointure et de tout tri global.
 - Un plan se lit de bas en haut. FileScan avec ses filtres poussés, Exchange pour le brassage, et deux HashAggregate autour de lui quand l'agrégat est bien écrit.
 - Spark écrit un dossier de fichiers, pas un fichier, et en snappy par défaut : 24 Mo là où le zstd du Cours 02 en écrit 14, pour le même contenu.
 - Les trois pièges coûteux sont le brassage inutile, la partition déséquilibrée et l'action qui ramène tout au pilote. Les trois se voient dans le plan ou dans les tailles de partition.
 - Sur l'historien complet de Boréal, une machine fait en 2,6 s ce que Spark fait en 52 à 80 s, avec les mêmes résultats. Le distribué se justifie par ce qui ne tient pas, jamais par principe.
-

Exercices

Les exercices de ce cours et leur corrigé vivent dans `Exercices_corriges_07.md`. Faites-les après le labo : plusieurs énoncés supposent que les deux traitements ont tourné, que les durées sont sous vos yeux et que les deux jeux de résultats existent encore.